ESS: Programming with Python

Python Basics

Abate Alemayehu

Ethiopian Statistical Service (ESS)

April 20–30, 2026

Email: abatea72@gmail.com

Overview

Sessions

- Session 1: Setting Up Python Environment (Slides 1-10)
- Session 2: Types and Variables (Slides 11-25)
- Session 3: Python Data Structures (Slides 26-45)
- Session 4: Python Functions (Slides 46-65)
- Session 5: Object-Oriented Programming in Python (Slides 66-85)

Info

By the end of this session, you will be able to write basic Python programs using variables, data structures, functions, and classes.

Slide 1: What is Python?

Key Characteristics

- High-level, interpreted programming language
- Designed for readability and simplicity
- Used in web development, data science, AI, research

"Python is the second-best language for everything"

Slide 2: Why Python for Research?

- **Free and open-source** — No licensing costs
- **Huge ecosystem** — Libraries for every scientific domain
- **Large community support** — Easy to find help
- **Cross-platform** — Windows, Mac, Linux

Slide 3: The Scientific Python Ecosystem

Library	Purpose
NumPy	numerical computing
Pandas	data analysis
Matplotlib	visualization
SciPy	advanced scientific computing
Jupyter	interactive notebooks

Info

All these libraries are included in the Anaconda distribution.

Slide 4: What is Anaconda?

Anaconda Features

- All-in-one Python distribution
- Includes Python + 250+ scientific packages
- Includes conda (package manager)
- No admin rights needed for installation

Slide 5: Installing Anaconda on Windows

1. Download Python 3.x installer from anaconda.com
2. Run installer as administrator
3. Check "Add Anaconda to my PATH"
4. Check "Make Anaconda the default Python"

Slide 6: Installing Anaconda on Mac

1. Download macOS installer
2. Double-click to install
3. Follow default settings
4. Anaconda Navigator opens after installation

Slide 7: Installing Anaconda on Linux

1. Download shell script (.sh file)
2. Open terminal: `bash Anaconda3-*.sh`
3. Follow prompts (yes to license, yes to PATH)
4. Close and reopen terminal

Slide 8: What is Jupyter Notebook?

Features

- Web-based interactive environment
- Combine code, text, equations, and visualizations
- Great for teaching and exploration
- Runs in browser (no internet needed)

Slide 9: Launching Jupyter Notebook

Two Ways to Launch

1. Open Anaconda Navigator → Launch Jupyter Notebook
2. OR terminal: `jupyter notebook`

After Launch

- Browser opens with file browser
- Click "New" → "Python 3"

Slide 10: Jupyter Notebook Interface

Key Concepts

- Cells: code or markdown
- Run cell: Shift + Enter
- Keyboard shortcuts (Esc for command mode, Enter for edit mode)
- Save checkpoint, restart kernel, shutdown

Info

Practice creating your first notebook with a simple print statement!

Slide 11: What is a Variable?

Definition

- Container for storing data
- Name that points to a value
- No need to declare type (dynamic typing)

```
1 # Example
2 x = 5
3 name = "Marta"
```

Slide 12: Variable Naming Rules

Valid Names

- `my_var`
- `var123`
- `_private`
- `student_name`

Invalid Names

- `123var` (starts with number)
- `my-var` (hyphen not allowed)
- `class` (reserved keyword)

Info

Best practice: Use lowercase with underscores (snake_case). Case-sensitive (age != Age).

Slide 13: Integers (int)

Properties

- Whole numbers (positive, negative, zero)
- No decimal point
- Unlimited precision

```
1 # Examples
2 age = 25
3 temperature = -10
4 large_number = 1000000
```

Slide 14: Floats (float)

Properties

- Decimal numbers
- Scientific notation possible
- Limited precision (floating-point error)

```
1 # Examples
2 pi = 3.14159
3 small = -0.001
4 scientific = 2.5e-4  # 0.00025
```

Slide 15: Strings (str)

Properties

- Sequence of characters
- Single quotes: 'Hello'
- Double quotes: "World"
- Triple quotes: '''Multi-line'''

```
1 # Examples
2 name = "Biruk"
3 greeting = 'Hello'
4 multi_line = """This spans
5 multiple lines"""
6 concatenated = "Hi" + " there"
```

Slide 16: Booleans (bool)

Properties

- Two values: `True` and `False`
- Case-sensitive (capital T and F)
- Result of comparisons: `5 > 3` → `True`
- Used in conditions and logic

```
# Examples
is_valid = True
is_empty = False
result = 10 > 5   # True
```

Slide 17: Type Conversion (Casting)

Conversion Functions

- `int()` → converts to integer
- `float()` → converts to float
- `str()` → converts to string
- `bool()` → converts to boolean

```
# Examples
int("42")       # 42
float("3.14")   # 3.14
str(100)        # "100"
bool(0)         # False
```

Slide 18: The print() Function

Usage

- Outputs text or variables to console
- Multiple items separated by commas
- end parameter to change line ending

```
1 # Examples
2 print("Hello, World!")
3 print("Hello", name)
4 print("No newline", end="")
```

Slide 19: The input() Function

Usage

- Takes user input from keyboard
- Always returns a string
- Can display a prompt
- Convert with `int()` or `float()` if needed

```
# Example
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

Slide 20: Comments in Python

Types of Comments

- Single-line: `# This is a comment`
- Multi-line: triple quotes (not official but works)
- Used to explain code
- Ignored by Python interpreter

```
# This is a single-line comment
x = 5  # Comment after code

"""
This is a multi-line
comment using triple quotes
"""
```

Slide 21: Basic Arithmetic Operators

Operator	Operation
+	addition
-	subtraction
*	multiplication
/	division (returns float)
//	integer division
%	modulus (remainder)
**	exponentiation

Slide 22: Comparison Operators

Operator	Meaning
==	equal to
!=	not equal to
>	greater than
<	less than
>=	greater than or equal
<=	less than or equal

Slide 23: Logical Operators

Operator	Behavior
and	True only if both are True
or	True if at least one is True
not	Reverses boolean value

Examples

- `(x > 0) and (x < 10)`
- `not (y == 0)`

Slide 24: Operator Precedence

1. Parentheses `()`
2. Exponentiation `**`
3. Multiplication/Division `* / // %`
4. Addition/Subtraction `+ -`
5. Comparisons `== != > < >= <=`
6. `not`
7. `and`
8. `or`

Info

Use parentheses to make complex expressions clear!

Slide 25: Code Example: Simple Calculator

```
1 name = input("Enter your name: ")
2 age = int(input("Enter your age: "))
3 years_to_100 = 100 - age
4 print(f"{name}, you will be 100 in {years_to_100} years")
```

Info

Try this code in Jupyter Notebook!

Slide 26: What are Data Structures?

Definition

- Ways to organize and store multiple values
- Each has different properties and use cases
- Built-in: list, tuple, dict, set

Slide 27: Lists – Definition

Properties

- Ordered, mutable collection
- Created with square brackets: `[1, 2, 3]`
- Can hold mixed types: `[1, "hello", 3.14]`
- Indexed starting at 0

```
1 fruits = ["apple", "banana", "cherry"]
2 mixed = [1, "hello", 3.14, True]
```

Slide 28: List Operations – Accessing

```
my_list = [10, 20, 30, 40, 50]

my_list[0]      # 10 (first element)
my_list[-1]     # 50 (last element)
my_list[1:3]    # [20, 30] (indices 1 to 2)
my_list[:3]     # [10, 20, 30] (first three)
my_list[2:]     # [30, 40, 50] (from index 2)
```

Slide 29: List Operations – Modifying

```
my_list = [1, 2, 3]

my_list.append(4)       # [1, 2, 3, 4]
my_list.insert(1, 10)   # [1, 10, 2, 3, 4]
my_list.remove(2)       # [1, 10, 3, 4]
my_list.pop()           # removes and returns 4
my_list[0] = 99         # [99, 10, 3]
```

Slide 30: List Operations – Other

```
my_list = [3, 1, 4, 1, 5, 9]

len(my_list)              # 6
my_list.sort()            # [1, 1, 3, 4, 5, 9]
sorted(my_list)           # returns new sorted list
sum(my_list)              # 23
max(my_list)              # 9
min(my_list)              # 1
5 in my_list              # True
```

Slide 31: List Comprehensions

Syntax

```
[expression for item in iterable if condition]
```

```python
# Square numbers 0-9
squares = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Filter positive numbers
positives = [x for x in numbers if x > 0]

# With condition
evens = [x for x in range(20) if x % 2 == 0]
```

Slide 32: Tuples – Definition

Properties

- Ordered, immutable collection
- Created with parentheses: `(1, 2, 3)`
- Single element: `(5,)` (comma needed!)
- Cannot change after creation

```
coordinates = (10, 20)
single = (5,)
empty = ()
```

Slide 33: When to Use Tuples

- Fixed data that shouldn't change (e.g., coordinates, RGB colors)
- Dictionary keys (lists cannot be keys)
- Function returning multiple values
- Faster than lists

Info

Use tuples when you want to ensure data doesn't change accidentally!

Slide 34: Dictionaries – Definition

Properties

- Key-value pairs
- Created with curly braces: `{"name": "Alice", "age": 25}`
- Keys must be immutable (string, int, tuple)
- Values can be any type

```
person = {
    "name": "Alice",
    "age": 25,
    "city": "Addis"
}
```

Slide 35: Dictionary Operations

```
person = {"name": "Alice", "age": 25}

# Access
person["name"]           # "Alice"

# Add/change
person["age"] = 26
person["city"] = "Addis"

# Delete
del person["age"]

# Check
"name" in person         # True

# Get all keys, values, items
```

Slide 36: Dictionary Methods

```
# Safe access with default
person.get("email", "unknown")  # "unknown"

# Merge dictionaries
person.update({"email": "alice@example.com", "phone": "12345"})

# Remove and return value
email = person.pop("email")

# Clear all items
person.clear()
```

Slide 37: Sets – Definition

Properties

- Unordered collection of unique elements
- Created with `set()` or curly braces: `{1, 2, 3}`
- No duplicates allowed
- Cannot index (no order)

```
colors = {"red", "green", "blue"}
unique = set([1, 2, 2, 3, 3, 3])  # {1, 2, 3}
```

Slide 38: Set Operations

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# Basic operations
A.add(7)            # {1,2,3,4,7}
A.remove(2)         # {1,3,4,7}

# Mathematical operations
A | B    # Union: {1,2,3,4,5,6}
A & B    # Intersection: {3,4}
A - B    # Difference: {1,2}
A ^ B    # Symmetric difference: {1,2,5,6}
```

Slide 39: When to Use Sets

- Remove duplicates from a list
- Membership testing (faster than lists)
- Mathematical set operations (union, intersection, etc.)

Info

Example: `unique_names = set(list_of_names)`

Slide 40: Choosing the Right Structure

Structure	Ordered	Mutable	Duplicates	Key-Value
List	Yes	Yes	Yes	No
Tuple	Yes	No	Yes	No
Dict	Yes (3.7+)	Yes	Keys: No	Yes
Set	No	Yes	No	No

Slide 41: Nested Data Structures

```
# Lists inside lists (matrix)
matrix = [[1, 2], [3, 4]]

# Lists inside dictionaries
person = {
    "name": "Ali",
    "grades": [85, 90, 78]
}

# Dictionaries inside lists
students = [
    {"name": "Ali", "grade": 85},
    {"name": "Bekele", "grade": 92}
]
```

Slide 42: Iterating Over Lists

```
my_list = [10, 20, 30]

# Method 1: direct item
for item in my_list:
    print(item)

# Method 2: by index
for i in range(len(my_list)):
    print(my_list[i])

# Method 3: with index and value
for index, value in enumerate(my_list):
    print(f"Index {index}: {value}")
```

Slide 43: Iterating Over Dictionaries

```python
person = {"name": "Almaz", "age": 28, "city": "Addis"}

# Iterate over keys
for key in person:
    print(key)

# Iterate over values
for value in person.values():
    print(value)

# Iterate over key-value pairs
for key, value in person.items():
    print(f"{key}: {value}")
```

Slide 44: Copying Data Structures

```
# Assignment copies reference (not value)
list1 = [1, 2, 3]
list2 = list1
list2[0] = 99  # list1 also changes!

# Shallow copy
list3 = list1.copy()
list4 = list(list1)

# Deep copy for nested structures
import copy
nested = [[1, 2], [3, 4]]
deep_copy = copy.deepcopy(nested)
```

Slide 45: Code Example: Student Grades

```python
students = [
    {"name": "Almaz", "grades": [85, 90, 78]},
    {"name": "Biruk", "grades": [92, 88, 84]}
]

for student in students:
    avg = sum(student["grades"]) / len(student["grades"])
    print(f"{student['name']}: {avg:.2f}")

# Output:
# Almaz: 84.33
# Biruk: 88.00
```

Slide 46: What is a Function?

Definition

- Reusable block of code
- Takes inputs (parameters), returns output
- Avoids code duplication
- Makes programs modular

Slide 47: Defining a Function

```
1 def function_name(parameter1, parameter2):
2     """Docstring: explains what function does"""
3     # code block
4     return result
```

Info

The def keyword starts a function definition.

Slide 48: Calling a Function

```python
def greet(name):
    return f"Hello, {name}!"

# Calling the function
message = greet("Biruk")
print(message)  # Hello, Biruk!
```

Key Points

- Arguments are passed to parameters
- Order matters (positional arguments)

Slide 49: Return Statement

```python
# Function with return
def add(a, b):
    return a + b

result = add(3, 5)  # result = 8

# Function without return (returns None)
def say_hello(name):
    print(f"Hi {name}")

output = say_hello("Alice")  # output = None

# Returning multiple values
def get_coordinates():
    return 10, 20
```

Slide 50: Docstrings

```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Parameters:
    length: float - length of rectangle
    width: float - width of rectangle

    Returns:
    float - area of rectangle
    """
    return length * width

# View docstring
help(calculate_area)
```

Slide 51: Default Arguments

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

# Using default
greet("Alice")              # "Hello, Alice!"

# Overriding default
greet("Bob", "Hi")          # "Hi, Bob!"
greet("Charlie", greeting="Hey")  # "Hey, Charlie!"
```

Warning

Default arguments are evaluated once at definition. Be careful with mutable defaults like lists!

Slide 52: Keyword Arguments

```
def describe_person(name, age, city):
    return f"{name} is {age} years old from {city}"

# Positional arguments (order matters)
describe_person("Alice", 25, "Addis")

# Keyword arguments (order doesn't matter)
describe_person(city="Hawassa", name="Bob", age=30)

# Mixed (positional first, then keyword)
describe_person("Charlie", city="Bahir Dar", age=35)
```

Slide 53: *args (Variable Positional Arguments)

```python
def sum_all(*args):
    """Sum any number of arguments"""
    return sum(args)

sum_all(1, 2, 3)       # 6
sum_all(10, 20, 30, 40)  # 100
sum_all()              # 0

def print_args(*args):
    for i, arg in enumerate(args):
        print(f"arg{i}: {arg}")
```

Slide 54: **kwargs (Variable Keyword Arguments)

```python
def print_info(**kwargs):
    """Print any number of keyword arguments"""
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=25, city="Addis")
# Output:
# name: Alice
# age: 25
# city: Addis

def func(a, b, *args, **kwargs):
    print(f"a={a}, b={b}")
    print(f"args={args}")
    print(f"kwargs={kwargs}")
```

Slide 55: Scope – Local Variables

```
def my_function():
    x = 10  # Local variable
    print(x)

my_function()  # Prints 10
# print(x)  # ERROR! x is not defined outside
```

Info

Local variables exist only inside the function where they are created.

Slide 56: Scope – Global Variables

```
x = 10  # Global variable

def read_global():
    print(x)  # Can read global

def modify_global():
    global x  # Need global keyword to modify
    x = 20

read_global()   # 10
modify_global()
print(x)        # 20
```

Warning

Avoid modifying global variables. Pass them as parameters instead!

Slide 57: The global Keyword

```
counter = 0

def increment():
    global counter
    counter += 1

def decrement():
    global counter
    counter -= 1

increment()
increment()
print(counter)  # 2
decrement()
print(counter)  # 1
```

Slide 58: Lambda Functions

```
# Anonymous, one-line functions
square = lambda x: x ** 2
square(5)    # 25

# Multiple parameters
add = lambda a, b: a + b
add(3, 5)    # 8

# No return statement needed
# Can be used anywhere a function is expected

# Often used with map, filter, sort
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, numbers))
```

Slide 59: Map Function

```python
# Apply function to every item in iterable
numbers = [1, 2, 3, 4, 5]

# Using lambda
squared = list(map(lambda x: x**2, numbers))
# [1, 4, 9, 16, 25]

# Using named function
def double(x):
    return x * 2

doubled = list(map(double, numbers))
# [2, 4, 6, 8, 10]

# Multiple iterables
list(map(lambda a, b: a + b, [1,2,3], [4,5,6]))
```

Slide 60: Filter Function

```python
# Filter items based on condition
numbers = [1, -2, 3, -4, 5, 0, -6]

# Keep positive numbers
positives = list(filter(lambda x: x > 0, numbers))
# [1, 3, 5]

# Keep even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
# [-2, -4, 0, -6]

# Using named function
def is_positive(x):
    return x > 0

positives = list(filter(is_positive, numbers))
```

Slide 61: Recursion

Definition

- Function that calls itself
- Must have base case (stop condition)
- Python has recursion limit (default 1000)

Info

Example problems: factorial, Fibonacci, tree traversal

Slide 62: Recursion Example

```
1  def factorial(n):
2      """Calculate n! recursively"""
3      # Base case
4      if n == 0:
5          return 1
6      # Recursive case
7      else:
8          return n * factorial(n - 1)
9
10 # Trace: factorial(4)
11 # = 4 * factorial(3)
12 # = 4 * 3 * factorial(2)
13 # = 4 * 3 * 2 * factorial(1)
14 # = 4 * 3 * 2 * 1 * factorial(0)
15 # = 4 * 3 * 2 * 1 * 1 = 24
```

Slide 63: Function as First-Class Object

```
# Assign function to variable
def greet():
    return "Hello"

say_hello = greet
print(say_hello())  # "Hello"

# Pass function as argument
def apply(func, value):
    return func(value)

apply(lambda x: x**2, 5)  # 25

# Return function from function
def make_multiplier(n):
    return lambda x: x * n
```

Slide 64: Decorators (Intro)

```python
# Functions that modify other functions
def timer(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        print(f"Time: {time.time() - start:.4f}s")
        return result
    return wrapper

@timer
def slow_function():
    sum(range(1000000))

slow_function()  # Prints execution time
```

Slide 65: Code Example: Temperature Converter

```python
def celsius_to_fahrenheit(c):
    return (c * 9/5) + 32

def fahrenheit_to_celsius(f):
    return (f - 32) * 5/9

def convert(value, from_unit, to_unit):
    if from_unit == "C" and to_unit == "F":
        return celsius_to_fahrenheit(value)
    elif from_unit == "F" and to_unit == "C":
        return fahrenheit_to_celsius(value)
    else:
        return value

print(convert(100, "C", "F"))  # 212.0
print(convert(212, "F", "C"))  # 100.0
```

Slide 66: What is OOP?

Definition

- Programming paradigm based on "objects"
- Objects contain data (attributes) and behavior (methods)
- Models real-world entities
- Languages: Python, Java, C++, C#

Slide 67: Classes vs Objects

Key Distinction

- **Class:** Blueprint or template
- **Object:** Instance of a class
- Example: Car is class, my_car is object
- One class can create many objects

Info

Class → Blueprint Object → Actual thing

Slide 68: Defining a Class

```
class ClassName:
    """Docstring: describes the class"""
    # class body
    pass

# Example
class Car:
    """Represents a car"""
    pass
```

Slide 69: Creating an Object (Instance)

```
class Car:
    pass

# Creating objects (instances)
my_car = Car()
your_car = Car()

# Each object is independent
print(my_car)   # <__main__.Car object at 0x...>
print(your_car) # <__main__.Car object at 0x...>
```

Info

Each object gets its own memory space.

Slide 70: Attributes

```python
class Car:
    pass

my_car = Car()

# Add attributes dynamically
my_car.brand = "Toyota"
my_car.color = "red"
my_car.year = 2020

# Access attributes
print(my_car.brand)  # Toyota
print(my_car.color)  # red

# Change attributes
my_car.year = 2021
```

Slide 71: Methods

```
1  class Car:
2      def start(self):
3          print("Engine started")
4
5      def stop(self):
6          print("Engine stopped")
7
8      def honk(self):
9          print("Beep beep!")
10
11 my_car = Car()
12 my_car.start()  # Engine started
13 my_car.honk()   # Beep beep!
14 my_car.stop()   # Engine stopped
```

Slide 72: The __init__ Method (Constructor)

```python
class Car:
    def __init__(self, brand, color, year):
        """Constructor - called when object is created"""
        self.brand = brand
        self.color = color
        self.year = year

    def display_info(self):
        print(f"{self.year} {self.color} {self.brand}")

# Creating objects with initial values
car1 = Car("Toyota", "red", 2020)
car2 = Car("Honda", "blue", 2021)

car1.display_info()  # 2020 red Toyota
car2.display_info()  # 2021 blue Honda
```

Slide 73: The self Parameter

```python
class Person:
    def __init__(self, name, age):
        self.name = name  # self refers to current instance
        self.age = age

    def introduce(self):
        print(f"Hi, I'm {self.name}")

# self is automatically passed by Python
person = Person("Almaz", 28)
person.introduce()  # self = person automatically

# You can see self if you call the method differently
Person.introduce(person)  # Same as person.introduce()
```

Slide 74: Class Example: Person

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"Hi, I'm {self.name} and I'm {self.age} years
            old")

    def have_birthday(self):
        self.age += 1
        print(f"{self.name} is now {self.age}")
```

Slide 75: Using the Person Class

```
# Create objects
person1 = Person("Almaz", 28)
person2 = Person("Biruk", 32)

# Use methods
person1.introduce()  # Hi, I'm Almaz and I'm 28 years old
person2.introduce()  # Hi, I'm Biruk and I'm 32 years old

# Modify object
person1.have_birthday()  # Almaz is now 29

# Access attributes directly
print(person1.name)  # Almaz
print(person2.age)   # 32
```

Slide 76: Class Attributes vs Instance Attributes

Class Attribute

- Shared by all instances
- Defined in class body (outside `__init__`)
- Same value for all objects

Instance Attribute

- Unique to each instance
- Defined in `__init__` using `self`
- Can have different values per object

Slide 77: Inheritance

Definition

- Create new class based on existing class
- Child class inherits attributes and methods
- Parent class (base/superclass)
- Child class (derived/subclass)

Info

"is-a" relationship: A Student is-a Person

Slide 78: Inheritance Syntax

```
class ParentClass:
    # parent class body
    pass

class ChildClass(ParentClass):
    # child class body
    pass
```

Info

Child class inherits all methods and attributes from parent.

Slide 79: Inheritance Example

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"Hi, I'm {self.name}")

class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)  # Call parent constructor
        self.student_id = student_id

    def study(self):
        print(f"{self.name} is studying")
```

Slide 80: The super() Function

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call parent constructor
        self.breed = breed

    def speak(self):
        super().speak()  # Call parent method
        print(f"{self.name} barks!")
```

Slide 81: Method Overriding

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Cat(Animal):
    def speak(self):  # Overrides parent method
        print("Meow!")

class Dog(Animal):
    def speak(self):  # Overrides parent method
        print("Woof!")

animals = [Animal(), Cat(), Dog()]
for animal in animals:
    animal.speak()
# Animal makes a sound
```

Slide 82: Encapsulation

Definition

- Bundling data and methods together
- Hiding internal state
- Use _ (single underscore) for protected
- Use __ (double underscore) for private

Info

Encapsulation helps prevent accidental modification of data.

Slide 83: Properties (@property)

Purpose

- Allows controlled access to attributes
- Getter: `@property`
- Setter: `@attribute.setter`
- Allows validation and computation

Slide 84: Property Example

```python
class BankAccount:
    def __init__(self, balance):
        self._balance = balance  # Protected attribute

    @property
    def balance(self):
        """Getter - returns balance"""
        return self._balance

    @balance.setter
    def balance(self, value):
        """Setter - validates before setting"""
        if value < 0:
            raise ValueError("Balance cannot be negative")
        self._balance = value
```

Slide 85: OOP Best Practices

Guidelines

- One class per file (for larger projects)
- Use meaningful class and method names
- Keep classes focused (single responsibility)
- Use inheritance where appropriate (is-a relationship)
- Composition over inheritance (has-a relationship)
- Document your classes with docstrings

End of session 1

Questions?

Topics Covered:

- Setting Up Python Environment
- Types and Variables
- Python Data Structures
- Python Functions
- Object-Oriented Programming

See you next for session 2: NumPy!